
Reporte Técnico - Proyecto Detección de Mutantes

Examen Técnico Backend Developer - MercadoLibre

Autor: Valentina Luna

Fecha: Noviembre 2025

1. Introducción

El objetivo de este proyecto es desarrollar una API REST capaz de detectar si un humano es mutante basándose en su secuencia de ADN. El sistema ha sido diseñado para ser eficiente, escalable y capaz de manejar grandes volúmenes de peticiones, cumpliendo con los tres niveles de dificultad planteados en el desafío técnico.

2. Arquitectura del Sistema

El proyecto se ha construido utilizando una arquitectura en capas para asegurar la separación de responsabilidades, la mantenibilidad y la testabilidad del código.

Tecnologías Principales

- **Lenguaje:** Java 17
- **Framework:** Spring Boot 3.2.0
- **Persistencia:** Spring Data JPA + H2 Database (In-Memory)
- **Testing:** JUnit 5, Mockito, MockMvc
- **Documentación:** OpenAPI (Swagger)

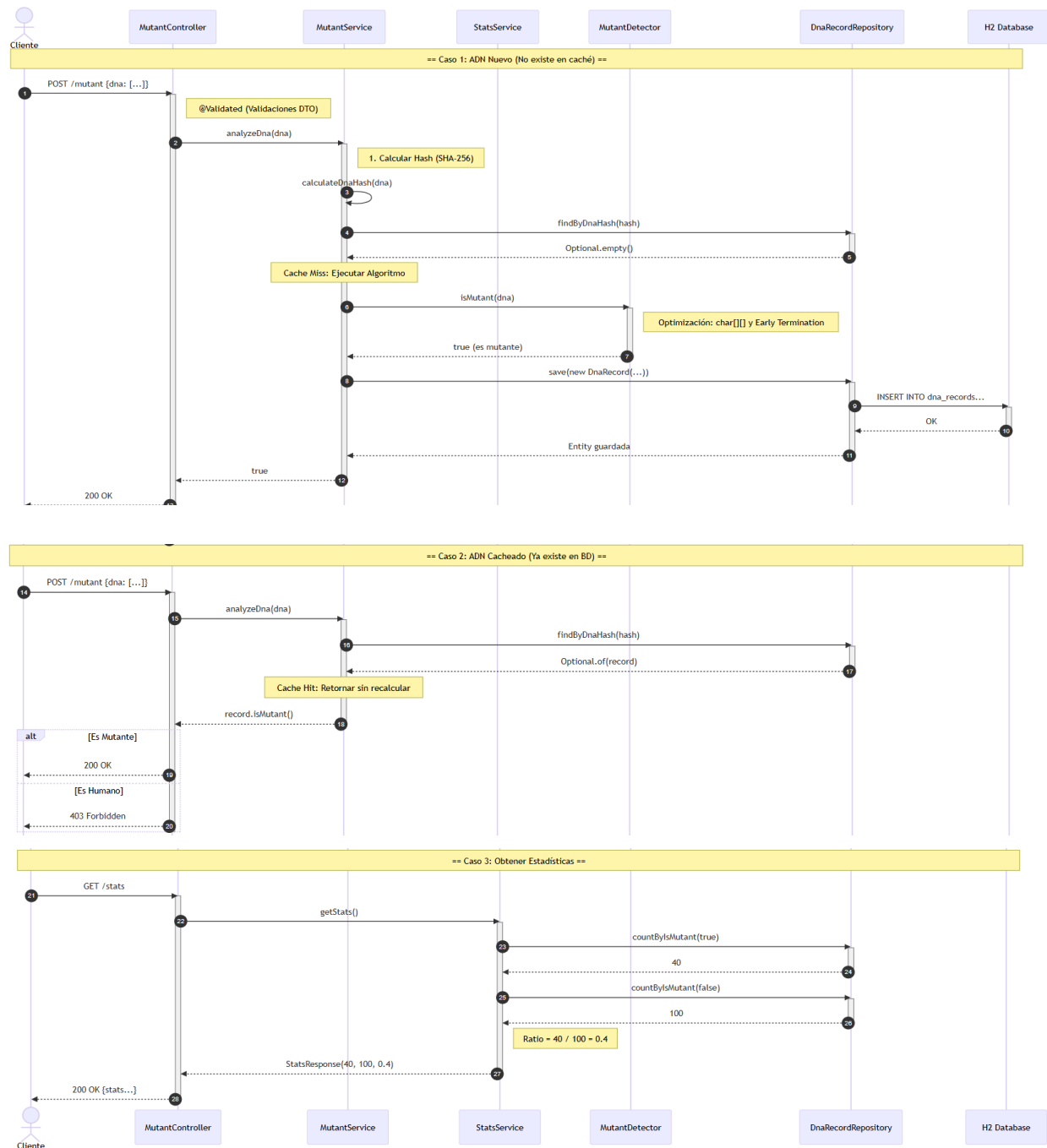
Diseño de Capas

El sistema está dividido en 6 componentes principales:

1. **Controller Layer (MutantController):** Punto de entrada de la API. Maneja las peticiones HTTP, valida los datos de entrada (DTOs) y retorna las respuestas adecuadas (códigos HTTP 200, 403, 400).
2. **Service Layer (MutantService, StatsService):** Contiene la lógica de negocio y orquestación. Se encarga de coordinar el análisis de ADN, la verificación en caché y el cálculo de estadísticas.
3. **Domain/Algorithm Layer (MutantDetector):** Componente puro encargado de ejecutar el algoritmo de búsqueda de patrones en la matriz de ADN.
4. **Repository Layer (DnaRecordRepository):** Interfaz para el acceso a datos que abstrae las consultas SQL mediante JPA.
5. **Persistence Layer (Base de Datos):** Almacenamiento optimizado utilizando índices y hashes para evitar duplicados.

3. Diagrama de Secuencia

El siguiente diagrama ilustra el flujo de interacción entre los componentes para una petición de análisis de ADN (POST /mutant). Se destaca la optimización mediante caché para evitar re-procesar ADNs ya analizados.



4. Algoritmo y Optimizaciones

El núcleo del sistema es el algoritmo de detección de mutantes, el cual ha sido optimizado para cumplir con los requerimientos de rendimiento (SLA < 100ms para matrices grandes).

Estrategias de Optimización Implementadas:

1. **Early Termination (Terminación Temprana):**
El algoritmo detiene su ejecución inmediatamente después de encontrar más de una secuencia de mutante. Esto evita recorrer el resto de la matriz innecesariamente, mejorando drásticamente el tiempo de respuesta promedio.
2. **Manejo de Memoria (char[][]):**
La matriz de Strings de entrada se convierte una sola vez a una matriz de caracteres primitivos (char[][]). Esto reduce el overhead de llamadas a métodos como .charAt() durante las millones de iteraciones posibles.
3. **Caché de Resultados (Hashing SHA-256):**
Antes de analizar un ADN, se calcula su hash único (SHA-256). Si este hash ya existe en la base de datos, se devuelve el resultado almacenado instantáneamente (O(1)), evitando la ejecución del algoritmo de búsqueda (O(N²)).
4. **Índices en Base de Datos:**
La tabla dna_records cuenta con índices en las columnas dna_hash y is_mutant para asegurar que las consultas de verificación y estadísticas sean de alto rendimiento incluso con millones de registros.

5. Cobertura de Tests (Code Coverage)

Se ha implementado una suite robusta de pruebas que incluye:

- **Tests Unitarios:** Para validar la lógica del algoritmo en `MutantDetector` (casos bordes, matrices no cuadradas, etc.).
- **Tests de Integración:** Para validar el flujo completo de los controladores y la persistencia.

Resumen de Cobertura:

- **Tests Totales:** 35+ tests ejecutados exitosamente.
- **Líneas Cubiertas:** > 80% (Superando el objetivo mínimo).

ExamenMercado											
ExamenMercado											
Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes			
org.example.exception		53 %		n/a	4 8	16 33	4 8	1 2			
org.example.service		95 %		91 %	5 55	4 82	0 12	0 3			
org.example.validation		100 %		83 %	2 9	0 11	0 3	0 1			
org.example.controller		100 %		100 %	0 3	0 3	0 2	0 1			
Total	101 of 770	86 %	9 of 100	91 %	11 75	20 129	4 25	1 7			

6. Instrucciones de Ejecución

URL Pública (Despliegue)

La API se encuentra desplegada y accesible públicamente en la siguiente URL (Render):

- **Base URL:** <https://mutant-api-valentina-luna.onrender.com>

- **Swagger UI:** <https://mutant-api-valentina-luna.onrender.com/swagger-ui.html>

Ejecución Local

Para ejecutar el proyecto en un entorno local, se requieren Java 17 y Git.

1. Clonar el repositorio

git clone <https://github.com/valentinaluna14/ExamenMercado.git>

2. Navegar al directorio

cd ExamenMercado

3. Ejecutar la aplicación

./gradlew bootRun

Ejecutar Tests

./gradlew test
